

Building a capstone application

COMPLETE LEARNING GUIDE

What You'll Learn

The capstone application is where every topic from Java Fundamentals through Advanced Concurrency comes together into one coherent project — a working piece of software you can walk an interviewer through, rather than isolated syntax knowledge. This guide covers how to plan and structure a capstone (a layered console or small web application, such as an inventory manager, a library system, or a task tracker backed by a real database), how to combine collections, generics, streams, JDBC, and concurrency deliberately rather than superficially, project organization with Maven or Gradle, the basics of writing JUnit tests for your core logic, sensible logging and error handling at the application level, and — critically for placement purposes — how to present the finished project confidently in an interview setting.

1. Choosing and Scoping a Capstone Project

A good capstone project is small enough to finish in the available time but rich enough to exercise most of the course's topics meaningfully — a CRUD-heavy domain (library system, inventory tracker, employee management, order processing) naturally exercises collections, JDBC persistence, and OOP design without requiring exotic algorithms. Scope it explicitly before writing code: list the core entities (e.g. Book, Member, Loan), the operations each supports, and one or two "stretch" features (like a report generated via streams, or a background task using the Executor framework) that demonstrate advanced material without becoming the whole project's focus.

2. Layered Architecture Applied to a Java Application

The same layered separation used in web backends applies directly to a standalone Java application: a data-access layer (JDBC repositories, one per entity, isolating all SQL) sits beneath a service layer (business rules — checking a book is actually available before creating a Loan), which is used by a presentation layer (a console menu, a REST controller, or a simple GUI). Keeping these layers strictly separated — the service layer never constructs SQL directly, the presentation layer never touches a Connection — is exactly what makes the project explainable in an interview: you can describe each layer's responsibility in one sentence.

JAVA

```
// Repository layer -- the only place SQL lives
public class BookRepository {
    public Optional<Book> findById(Connection conn, int id) throws SQLException
    { /* public void save(Connection conn, Book book) throws SQLException { /* ...
    */ }

// Service layer -- business rules, no SQL
public class LoanService {
    private final BookRepository books;
    private final LoanRepository loans;

    public Loan checkOut(int bookId, int memberId) {
        Book book = books.findById(...).orElseThrow(() -> new
        NoSuchElementException("Book not found"));
        if (!book.isAvailable()) throw new IllegalStateException("Book already
        checked out."); create and persist a Loan ...
    }
}
```

3. Combining Collections, Generics, and Streams for Real Logic

The strongest capstone projects use these tools purposefully, not decoratively: a generic `Repository<T, ID>` interface (implemented per entity) demonstrates real generics usage rather than a textbook `Box<T>` example; a report method that groups loans by member and counts them via `Collectors.groupingBy()` demonstrates practical stream usage over a hand-written loop with a `HashMap` accumulator; a `TreeMap` keeping overdue loans sorted by due date demonstrates a deliberate collection choice justified by a real requirement (needing sorted iteration) rather than an arbitrary one.

JAVA

```
public interface Repository<T, ID> {
    Optional<T> findById(ID id);
    List<T> findAll();
    T save(T entity);
    void deleteById(ID id);
}

// BookRepository implements Repository<Book, Integer> -- generics doing real
work

Map<Member, Long> overdueCountByMember = loans.stream()
    .filter(Loan::isOverdue)
    .collect(Collectors.groupingBy(Loan::getMember, Collectors.counting()));
```

4. Project Structure with Maven or Gradle

A capstone should use a standard build tool rather than manually compiling `.java` files — Maven (declarative XML, `pom.xml`, convention-heavy) or Gradle (Groovy/Kotlin DSL, `build.gradle`, more flexible) both manage dependencies (a JDBC driver, JUnit, a logging library) and enforce the standard source layout (`src/main/java`, `src/test/java`) that every Java tool and IDE expects. This matters for interviews specifically because "how do you manage dependencies and build this project" is a fair follow-up question, and a project with a proper `pom.xml/build.gradle` signals professional practice over a copy-pasted script.

5. Testing Core Logic with JUnit

The service layer — business rules with no direct database or I/O dependency — is exactly what unit tests should target first, since it can be tested in isolation without a real database connection (using a mock or in-memory implementation of the repository interfaces). A JUnit 5 test method is annotated `@Test`, uses `assertEquals/assertThrows/assertTrue` from `org.junit.jupiter.api.Assertions` to verify behavior, and each test should cover one specific behavior or edge case (checking out an already-checked-out book should throw, not silently succeed) rather than one giant test exercising the whole workflow at once.

JAVA

```
@Test
void checkOutThrowsWhenBookUnavailable() {
    Book book = new Book(1, "Effective Java", false); // not available
    when(bookRepository.findById(1)).thenReturn(Optional.of(book));

    assertThrows(IllegalStateException.class, () -> loanService.checkOut(1,
memberId));
}
```

6. Logging and Application-Level Error Handling

A capstone should use a real logging framework (SLF4J with Logback, the de facto Java standard) instead of `System.out.println()` for anything beyond quick debugging — proper logging carries severity levels (DEBUG, INFO, WARN, ERROR), timestamps, and can be redirected to a file, all of which `println()` cannot do. At the presentation layer, catch domain exceptions thrown by the service layer (`NoSuchElementException`, `IllegalStateException`, or custom exceptions like `BookUnavailableException`) and translate them into a clear user-facing message, rather than letting a raw stack trace print to the console or crash the application.

7. Presenting the Capstone in an Interview

When asked to walk through your capstone, lead with the problem it solves and its layered structure in 30 seconds before diving into any code, then be ready to justify specific decisions: why this collection type here, why this design pattern there, what you'd change with more time. Interviewers are often more interested in your reasoning and awareness of trade-offs than in the project's polish — being able to say "I used a `HashMap` here for $O(1)$ lookup, but if I needed sorted iteration I'd switch to a `TreeMap`" demonstrates exactly the depth this program is built to develop, and is far more valuable than a feature-complete project you can't explain.

Common Pitfalls

- Scoping the capstone too large to finish, leaving core features half-built instead of a smaller project that's fully working and well-tested.

- Mixing SQL directly into the service or presentation layer instead of keeping it isolated in a repository layer, making the project harder to explain and to test.
- Using generics, streams, or design patterns superficially just to "check a box" rather than because they solve a real problem in the project.
- Skipping tests entirely, then being unable to answer "how did you verify this works?" beyond "I ran it manually."
- Using `System.out.println()` for all output instead of a real logging framework, losing severity levels and making production-style debugging impossible.
- Catching exceptions in the presentation layer without translating them into a useful message, letting a raw stack trace confuse the end user.
- Being unable to justify a specific technical decision (why this collection, why this pattern) when asked in an interview, revealing the choice was arbitrary rather than deliberate.

Key Takeaways

- Scope the capstone to finish completely and be well-tested, rather than attempting every feature and finishing none of them.
- Apply the same layered architecture (repository / service / presentation) used in production backends, keeping SQL isolated from business logic.
- Use generics, collections, and streams because they solve a real problem in your project — be ready to justify every specific choice.
- A standard build tool (Maven or Gradle) and a proper `src/main/src/test` layout signal professional practice to an interviewer.
- Unit-test the service layer in isolation using mocked repositories — this is the layer most worth testing first.
- When presenting the project, lead with the problem and structure, then be ready to explain and justify specific technical trade-offs.

Learn More

Oracle Java Tutorials — Trail: Learning the Java Language

<https://docs.oracle.com/javase/tutorial/java/index.html>

JUnit 5 User Guide

<https://junit.org/junit5/docs/current/user-guide/>

Maven — Getting Started Guide

<https://maven.apache.org/guides/getting-started/index.html>